

Handout: Einführung in C (Teil 2)

1. Kontrollstrukturen: switch-case

Die `switch`-Anweisung dient dazu, eine Variable auf Gleichheit mit mehreren Konstanten zu prüfen.

- **Syntax:**

```
switch (WERT) {  
    case WERT_1:  
        // Anweisungen  
        break;  
    case WERT_2:  
        // Anweisungen  
        break;  
    default:  
        // Fallback, wenn kein case passt  
}
```

- **Wichtige Regeln:**

- `WERT` in der Klammer muss ein **Ganzzahl-Typ** (z. B. `int`) sein.
- `case`-Werte müssen **Konstanten** sein.
- **break:** Verlässt den `switch`-Block. Ohne `break` wird der Code des nächsten `case` einfach mit ausgeführt ("Fall-through").
- **default:** Ein optionaler Zweig, der ausgeführt wird, wenn keine der `case`-Bedingungen zutrifft.

2. Operatoren

Ein Operator verbindet oder transformiert Operanden.

Arten von Operatoren

Typ	Anzahl Operanden	Beispiel
unär	1	<code>!a</code> (Negation)
binär	2	<code>a + b</code> (Addition)
ternär	3	<code>a ? b : c</code> (Bedingung: wenn <code>a</code> wahr, dann <code>b</code> , sonst <code>c</code>)

Position der Operatoren

- **Prefix:** Operator steht vor dem Operanden (`!a`).
- **Infix:** Operator steht zwischen den Operanden (`a + b`).
- **Postfix:** Operator steht hinter dem Operanden (`a++`).

Assoziativität

- **Linksassoziativ:** Auswertung von links nach rechts (z. B. `a + b - c` $\rightarrow$ `(a + b) - c`).
- **Rechtsassoziativ:** Auswertung von rechts nach links (z. B. Zuweisungen: `a = b = c` $\rightarrow$ `a = (b = c)`).

Increment, Decrement & Compound Assignment

Operator	Bezeichnung	Beispiel	Bedeutung
<code>++</code>	Postfix-Inkrement	<code>a++</code>	Gibt <code>a</code> zurück, erhöht a danach um 1
<code>++</code>	Präfix-Inkrement	<code>++a</code>	Erhöht <code>a</code> um 1, gibt das neue <code>a</code> zurück
<code>--</code>	Postfix-Dekrement	<code>a--</code>	Gibt <code>a</code> zurück, verringert a danach um 1
<code>--</code>	Präfix-Dekrement	<code>--a</code>	Verringert <code>a</code> um 1, gibt das neue <code>a</code> zurück
<code>+=</code>	Compound Assignment	<code>a += 2</code>	<code>a = a + 2</code>
<code>*=</code>	Compound Assignment	<code>a *= 2</code>	<code>a = a * 2</code>
<code>/=</code>	Compound Assignment	<code>a /= 2</code>	<code>a = a / 2</code> (Ganzzahldivision)
<code>%=</code>	Modulo Assignment	<code>a %= 2</code>	<code>a = a % 2</code> (Restwert)

3. Schleifen

Schleifen ermöglichen die Wiederholung von Codeblöcken (**Iteration**).

Schleife	Typ	Funktionsweise
while	kopfgesteuert	Prüft Bedingung vor der Ausführung.
do-while	fußgesteuert	Führt Code mindestens einmal aus, prüft Bedingung danach.
for	kopfgesteuert	Kombination aus Initialisierung; Bedingung; Update.

- **Endlosschleifen:**
 - `while (1) { ... }`
 - `do { ... } while (1);`
 - `for (;;) { ... }`
- **Schleifensteuerung:**
 - **break:** Bricht die gesamte Schleife sofort ab.
 - **continue:** Überspringt den Rest der aktuellen Iteration und springt zum nächsten Durchlauf (bei `while/for` zum Kopf, bei `do-while` zum Fuß).
- **Verschachtelte Schleifen:** Eine Schleife innerhalb einer anderen. Die innere Schleife durchläuft alle ihre Iterationen für jede einzelne Iteration der äußeren Schleife.

4. Arrays (Felder)

Ein Array ist eine zusammenhängende Sammlung von Elementen desselben Typs.

- **Eigenschaften:**
 - **0-basierte Indizierung:** Das erste Element ist `a[0]`, das letzte bei Größe n ist `a[n-1]`.
 - **Feste Länge:** Die Größe muss bei der Deklaration bekannt sein.
 - **Speicher:** Der Arrayname ist die Startadresse im Speicher.
- **Initialisierung:**
 - `int arr[3] = {1, 2, 3};` (Explizit)
 - `int arr[] = {1, 2};` (Compiler ermittelt Größe automatisch)
 - Nicht initialisierte lokale Arrays enthalten undefinierte (zufällige) Werte.
- **Größe berechnen:** `size_t n = sizeof(array) / sizeof(typ);`
- **Gefahr (Buffer Overflow):** Der Zugriff auf Indizes außerhalb des Bereichs (`index < 0` oder `index >= n`) führt zu **undefiniertem Verhalten** (oft Absturz/Segmentation Fault).
- **Mehrdimensionale Arrays:** `int a[2][3];` (z. B. für Tabellen/Matrizen).

5. Strings (Zeichenketten)

In C gibt es keinen eigenen Datentyp für Strings. Strings sind **nullterminierte** Zeichenketten. Strings können als **String-Literal**, als Array oder als Folge von Bytes im dynamischen Speicher vorliegen.

- **Adressierung von Zeichen:** Die Zeichen eines Strings können mit `[i]` wie bei Arrays angesprochen werden.
- **Das Nullzeichen (`\0`):** Markiert das Ende eines Strings im Speicher.
- **String-Literale:** `"Hallo"` liegt in einem unveränderlichen Speicherbereich als `{ 'H', 'a', 'l', 'l', 'o', '\0' }`.
- **Eingabe:**
 - `scanf("%s", buf);` → Stoppt bei Leerzeichen, Gefahr des Überlaufs.
 - `fgets(buf, sizeof(buf), stdin);` → **Sicherer**, liest bis `\n` oder Puffergrenze, inkludiert `\n`.
- **Wichtige Funktionen (`<string.h>`):**

Funktion	Zweck	Hinweis
<code>strlen(s)</code>	Länge des Strings	Liefert Länge ohne <code>\0</code> .
<code>strcmp(s1, s2)</code>	Vergleich	0 wenn gleich, <code><0</code> wenn <code>s1 < s2</code> , <code>>0</code> wenn <code>s1 > s2</code> .
<code>strcpy(dst, src)</code>	Kopieren	Gefährlich! Zielpuffer muss groß genug sein.
<code>strncpy(dst, src, n)</code>	Sicheres Kopieren	Kopiert maximal <code>n</code> Zeichen.
<code>strcspn(s, "\n")</code>	Suche nach Zeichen	Hilfreich, um <code>\n</code> nach <code>fgets</code> zu entfernen.

Wichtig: Ein Vergleich mit `==` (z. B. `str1 == str2`) vergleicht in C die **Speicheradressen**, nicht den Textinhalt! Nutze dafür immer `strcmp()`.